

Task Manager with User Authentication

Course-End Project — Write-up

Project	Task Manager with User Authentication
Language	Python 3 (standard library only)
Interface	Menu-driven command-line application
Persistence	JSON files (users.json, tasks.json)
Security	Salted PBKDF2-HMAC-SHA256 password hashing

1. Problem Statement

In today's world, individuals often need to keep track of various tasks in a structured way. This project builds a Task Manager that allows users to manage their tasks. The system includes user authentication, meaning each user has to log in with a username and password. Once logged in, users can create, view, update, and delete their tasks. Each user's tasks are stored separately, and only the authenticated user can access their own tasks.

Objectives

- 1 Design and implement a user authentication system (login and registration).
- 2 Create a task management system that allows users to add, view, mark as completed, and delete tasks.
- 3 Use file handling to store user credentials and tasks persistently.
- 4 Create an interactive menu-driven interface to manage tasks.

2. Design & Approach

2.1 Data storage

Two JSON files act as the persistence layer, replacing what would be database tables in a larger system:

- **users.json** — maps each username to a per-user random salt and a PBKDF2-HMAC-SHA256 hash of the password (100,000 iterations). The plaintext password is never written to disk.
- **tasks.json** — maps each username to a list of that user's tasks (*id*, *description*, *status*). Keying by username is what guarantees one user can never see another user's tasks.

2.2 Authentication flow

Registration rejects a username that already exists in users.json, then stores a freshly generated salt plus the resulting password hash. Login looks up the username, re-derives the hash from the entered password using the stored salt, and compares it to the stored hash using a constant-time comparison (`secrets.compare_digest`) to reduce timing-attack risk. A successful login returns the username to the main program, which then opens that user's task menu.

2.3 Task operations

- **Add** — assigns the next sequential task ID for that user and sets status to Pending.
- **View** — prints a formatted table of ID / status / description for the logged-in user only.
- **Mark completed** — looks up a task by ID and flips its status to Completed.
- **Delete** — looks up a task by ID and removes it from that user's list.

Every operation re-saves the affected JSON file immediately, so state survives an unexpected exit and there is no in-memory-only data.

2.4 Menu structure

A top-level menu (Register / Login / Exit) wraps an inner, per-session menu (Add / View / Mark Completed / Delete / Logout) that loops until the user logs out, at which point control returns to the top-level menu.

2.5 Error handling

- Empty username/password/description input is rejected with a message.
- Duplicate usernames are rejected at registration.
- Incorrect passwords are rejected at login without revealing whether the username exists or the password is what's wrong.
- Non-numeric or non-existent task IDs are caught with a clear message instead of crashing.

3. Screenshots — Sample Run

The following screenshots are captured from an actual terminal session running `python3 task_manager.py`, exercising every menu option end to end.

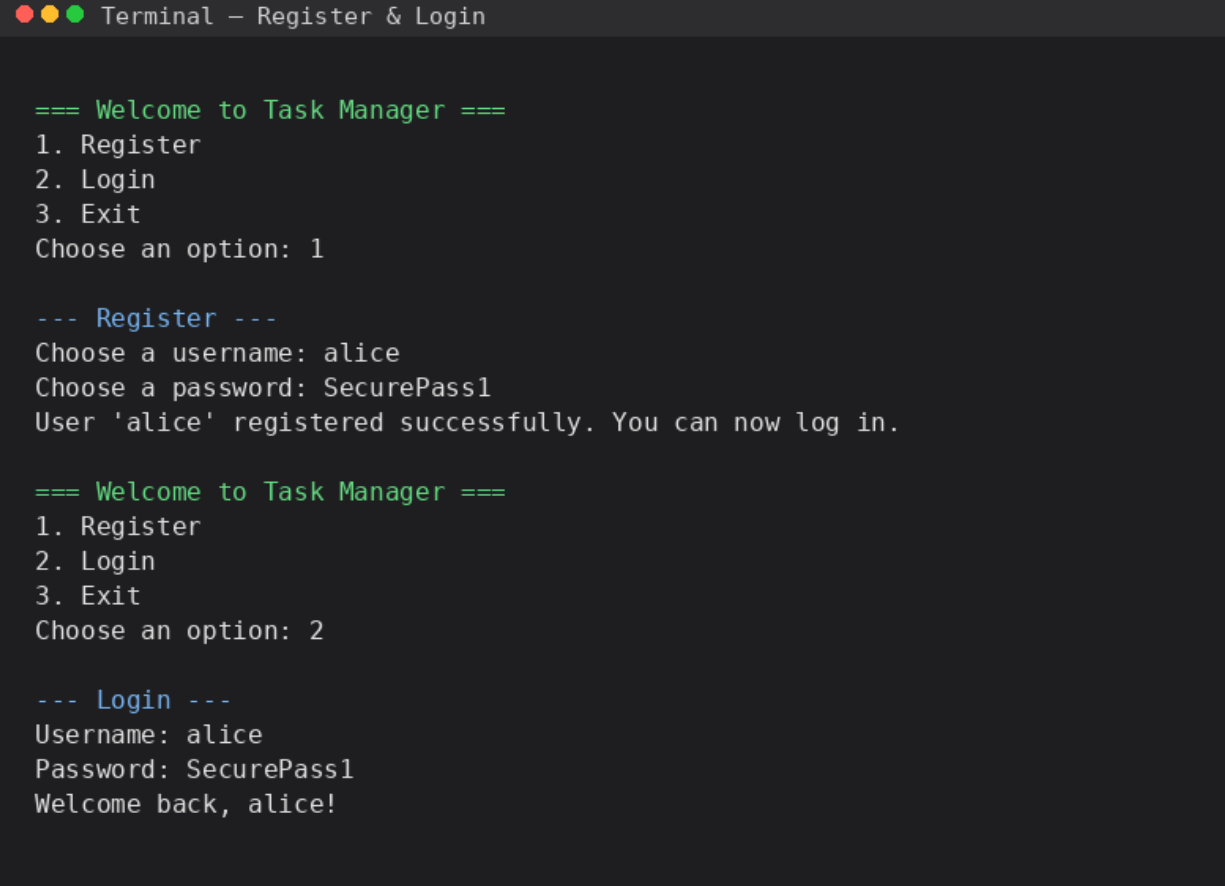
A terminal window titled "Terminal - Register & Login" with a dark background and light green text. The text shows a menu with three options: 1. Register, 2. Login, and 3. Exit. The user chooses option 1, leading to a registration screen. The user enters 'alice' as a username and 'SecurePass1' as a password. A confirmation message states 'User 'alice' registered successfully. You can now log in.' The user returns to the main menu and chooses option 2, leading to a login screen. The user enters 'alice' as the username and 'SecurePass1' as the password. A final message says 'Welcome back, alice!'.

Figure 1. Registering a new user (“alice”) and logging in.

```
Terminal - Add / View / Mark Completed

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 1
Enter task description: Buy groceries
Task added (ID 1).

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 1
Enter task description: Finish quarterly report
Task added (ID 2).

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 2

--- Tasks for alice ---
ID   Status      Description
-----
1    Pending     Buy groceries
2    Pending     Finish quarterly report

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 3

--- Tasks for alice ---
ID   Status      Description
-----
1    Pending     Buy groceries
2    Pending     Finish quarterly report
Enter the task ID to mark as completed: 1
Task 1 marked as Completed.
```

Figure 2. Adding two tasks, viewing the list, and marking a task completed.

```
Terminal - Delete Task, Logout & Exit

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 2

--- Tasks for alice ---
ID   Status      Description
-----
1    Completed   Buy groceries
2    Pending     Finish quarterly report

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 4

--- Tasks for alice ---
ID   Status      Description
-----
1    Completed   Buy groceries
2    Pending     Finish quarterly report
Enter the task ID to delete: 2
Task 2 deleted.

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 2

--- Tasks for alice ---
ID   Status      Description
-----
1    Completed   Buy groceries

=== Task Manager - logged in as alice ===
1. Add a Task
2. View Tasks
3. Mark a Task as Completed
4. Delete a Task
5. Logout
Choose an option: 5
Logging out alice. Goodbye!

=== Welcome to Task Manager ===
1. Register
2. Login
3. Exit
Choose an option: 3
Goodbye!
```

Figure 3. Viewing updated status, deleting a task, logging out, and exiting.

4. Source Code

Full source is provided separately as `task_manager.py` (also submitted alongside this write-up). It is reproduced below for reference.

```
"""
Task Manager with User Authentication
Course-End Project

A menu-driven, file-persisted task manager. Each user registers with a
username/password (password stored only as a salted SHA-256 hash), logs in,
and then manages their own tasks (add / view / mark completed / delete).
Every user's tasks are kept separate, so an authenticated user only ever
sees their own data.

Data files (created in the same directory as this script on first run):
  users.json -> { username: {"salt": "...", "password_hash": "..."} }
  tasks.json -> { username: [ {"id": 1, "description": "...", "status": "Pending"}, ... ] }
"""

import json
import os
import hashlib
import secrets

USERS_FILE = "users.json"
TASKS_FILE = "tasks.json"

# -----
# File handling helpers
# -----

def _load_json(path, default):
    """Load a JSON file, returning `default` if it doesn't exist or is empty/corrupt."""
    if not os.path.exists(path):
        return default
    try:
        with open(path, "r") as f:
            content = f.read().strip()
            if not content:
                return default
            return json.loads(content)
    except (json.JSONDecodeError, OSError):
        return default

def _save_json(path, data):
    with open(path, "w") as f:
        json.dump(data, f, indent=2)

def load_users():
    return _load_json(USERS_FILE, {})

def save_users(users):
    _save_json(USERS_FILE, users)

def load_tasks():
    return _load_json(TASKS_FILE, {})

def save_tasks(tasks):
```

```

    _save_json(TASKS_FILE, tasks)

# -----
# Password hashing
# -----

def hash_password(password, salt=None):
    """Return (salt, hash) using PBKDF2-HMAC-SHA256. A random salt is
    generated per user so identical passwords never produce identical
    stored hashes."""
    if salt is None:
        salt = secrets.token_hex(16)
    pw_hash = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt.encode("utf-8"), 100_000
    ).hex()
    return salt, pw_hash

def verify_password(password, salt, stored_hash):
    _, pw_hash = hash_password(password, salt)
    return secrets.compare_digest(pw_hash, stored_hash)

# -----
# User authentication
# -----

def register():
    """Prompt for a new username/password, enforce a unique username, and
    store a salted hash of the password (never the plaintext password)."""
    users = load_users()
    print("\n--- Register ---")
    username = input("Choose a username: ").strip()

    if not username:
        print("Username cannot be empty.")
        return
    if username in users:
        print(f"Username '{username}' is already taken. Please choose another.")
        return

    password = input("Choose a password: ").strip()
    if not password:
        print("Password cannot be empty.")
        return

    salt, pw_hash = hash_password(password)
    users[username] = {"salt": salt, "password_hash": pw_hash}
    save_users(users)

    # Give the new user an empty task list right away
    tasks = load_tasks()
    tasks.setdefault(username, [])
    save_tasks(tasks)

    print(f"User '{username}' registered successfully. You can now log in.")

def login():
    """Prompt for credentials and validate against stored (hashed) data.
    Returns the username on success, or None on failure."""
    users = load_users()
    print("\n--- Login ---")
    username = input("Username: ").strip()
    password = input("Password: ").strip()

    user_record = users.get(username)

```

```

    if user_record is None:
        print("No such user. Please register first.")
        return None

    if verify_password(password, user_record["salt"], user_record["password_hash"]):
        print(f"Welcome back, {username}!")
        return username

    print("Incorrect password.")
    return None

# -----
# Task management
# -----

def _next_task_id(user_tasks):
    if not user_tasks:
        return 1
    return max(task["id"] for task in user_tasks) + 1

def add_task(username):
    tasks = load_tasks()
    user_tasks = tasks.setdefault(username, [])

    description = input("Enter task description: ").strip()
    if not description:
        print("Task description cannot be empty. Task not added.")
        return

    task = {
        "id": _next_task_id(user_tasks),
        "description": description,
        "status": "Pending",
    }
    user_tasks.append(task)
    save_tasks(tasks)
    print(f"Task added (ID {task['id']}).")

def view_tasks(username):
    tasks = load_tasks()
    user_tasks = tasks.get(username, [])

    print(f"\n--- Tasks for {username} ---")
    if not user_tasks:
        print("You have no tasks yet.")
        return

    print(f"{'ID':<5}{'Status':<12}Description")
    print("-" * 40)

    for task in user_tasks:
        print(f"{task['id']:<5}{task['status']:<12}{task['description']}")

def _find_task(user_tasks, task_id):
    for task in user_tasks:
        if task["id"] == task_id:
            return task
    return None

def mark_completed(username):
    tasks = load_tasks()
    user_tasks = tasks.get(username, [])

```

```

    if not user_tasks:
        print("You have no tasks to update.")
        return

    view_tasks(username)
    raw_id = input("Enter the task ID to mark as completed: ").strip()
    if not raw_id.isdigit():
        print("Please enter a valid numeric task ID.")
        return

    task = _find_task(user_tasks, int(raw_id))
    if task is None:
        print(f"No task found with ID {raw_id}.")
        return

    if task["status"] == "Completed":
        print(f"Task {task['id']} is already marked as Completed.")
        return

    task["status"] = "Completed"
    save_tasks(tasks)
    print(f"Task {task['id']} marked as Completed.")

def delete_task(username):
    tasks = load_tasks()
    user_tasks = tasks.get(username, [])

    if not user_tasks:
        print("You have no tasks to delete.")
        return

    view_tasks(username)
    raw_id = input("Enter the task ID to delete: ").strip()
    if not raw_id.isdigit():
        print("Please enter a valid numeric task ID.")
        return

    task = _find_task(user_tasks, int(raw_id))
    if task is None:
        print(f"No task found with ID {raw_id}.")
        return

    user_tasks.remove(task)
    tasks[username] = user_tasks

    save_tasks(tasks)
    print(f"Task {task['id']} deleted.")

# -----
# Menus
# -----

def task_menu(username):
    """The logged-in menu loop: add / view / complete / delete / logout."""
    options = {
        "1": ("Add a Task", add_task),
        "2": ("View Tasks", view_tasks),
        "3": ("Mark a Task as Completed", mark_completed),
        "4": ("Delete a Task", delete_task),
    }

    while True:
        print(f"\n=== Task Manager – logged in as {username} ===")
        for key, (label, _) in options.items():
            print(f"{key}. {label}")
        print("5. Logout")

```

```

        choice = input("Choose an option: ").strip()

        if choice == "5":
            print(f"Logging out {username}. Goodbye!")
            return

        action = options.get(choice)
        if action is None:
            print("Invalid option. Please choose 1-5.")
            continue

        _, func = action
        func(username)

def main_menu():
    """Top-level menu: register, login, or exit the program."""
    while True:
        print("\n=== Welcome to Task Manager ===")
        print("1. Register")
        print("2. Login")
        print("3. Exit")

        choice = input("Choose an option: ").strip()

        if choice == "1":
            register()
        elif choice == "2":
            username = login()
            if username:
                task_menu(username)
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid option. Please choose 1-3.")

if __name__ == "__main__":
    main_menu()

```

5. Project Link & How to Run

This project was generated and tested in a local, offline sandbox with no internet access, so no hosted repository link (e.g. GitHub) could be created or verified as part of this deliverable. To publish it yourself:

- Create a new GitHub repository and add `task_manager.py` (e.g. `git init && git add task_manager.py && git commit -m "Task manager" && git push`), or
- Paste the script into a Replit / Google Colab / any online Python runner that supports interactive input () for a shareable live link.

Running locally

Requires Python 3.7+ and no external packages (uses only the standard library: json, os, hashlib, secrets).

- 1 Save `task_manager.py` to a folder.
- 2 Open a terminal in that folder.
- 3 Run: `python3 task_manager.py`
- 4 Follow the on-screen menu to register, log in, and manage tasks.

`users.json` and `tasks.json` are created automatically in the same folder on first run and persist between sessions.